

Required Capstone Checkpoint 3.1: Evaluation Infrastructure and Baseline Diagnosis

Directions

Download this worksheet and save a copy to your device before you begin working. Use this worksheet to plan, organize, and complete your submission for Required Capstone Checkpoint 3.1. Type your responses directly into the document, save your completed work, and upload the finished file when you are ready to submit.

Completion Checklist

- ☐ A clear description of the retrieval system and intended user tasks
- ☐ Defined and justified evaluation criteria, metrics, and testing approach
- ☐ Baseline evaluation results supported by relevant examples or evidence
- ☐ Identification of meaningful system strengths, weaknesses, and likely failure modes
- ☐ Evidence that the evaluation framework can detect degraded, manipulated, or otherwise problematic outputs
- ☐ Reflection on potential blind spots or limitations in the evaluation framework

Step 1. System Overview

Describe the baseline retrieval system you developed for your selected capstone scenario.

- Briefly explain what the system retrieves or answers.
- Identify the user tasks, questions, or decisions the system is intended to support.

Brief description of your retrieval system

The system is designed to answer questions on a corpus of 2, 419 Wikipedia articles in HTML format. These are written in English, and they cover significant people, places, and topics. The system should always generate answers based on the HTML files it is provided, never from its own internal training knowledge. It must never fabricate an answer or citation and must honestly decline to answer when the information required is not at hand. The baseline retrieval system uses simple text extraction from the HTML files to create an in-memory database of text entries and a persistent database of vector embeddings. It uses a hybrid retriever consisting of BM25 keyword search, semantic search using vector embeddings, and an algorithm to weight and merge results, which are then passed as whole article text documents to the LLM as context for the user's query.

Intended user tasks or questions

The system should handle common queries posed by users of Wikipedia, as documented by Singer, Lemmerich, West, Zia, Wulczyn, Strohmaier, and Leskovec in their foundational 2017 paper “Why We Read Wikipedia”, presented at the Proceedings of the 26th International Conference on World Wide Web, available at <https://arxiv.org/abs/1702.05379>.

This means handling the different dimensions of Singer and Lemmerich’s taxonomy, **including information need** (simple fact lookup, getting an overview of a topic, or getting an in-depth understanding of a topic), **motivation** (work or school assignment; making a personal decision such as buying a book or choosing a travel destination; getting information on a topic referenced in media such as TV, a book, a film; finding out about a topic that came up in conversation; satisfying boredom or random curiosity; or a desire for deep learning), and **prior knowledge** (familiar with the subject or learning about it for the first time). Each user scenario manifests itself differently in the way that questions are posed and what type of response is expected.

The system should be able to handle all these scenarios. It also needs to be able to handle problematic queries honestly and gracefully, such as when the question is ambiguous, is based on a false premise, or is simply unanswerable from the corpus, whether the information is missing entirely from the corpus or is stale or out-of-date.

Step 2. Evaluation Design

Define how you will evaluate the baseline system before introducing further optimization or architectural changes.

- Identify the criteria you will use to judge system performance.
- Select metrics that reflect your scenario, such as relevance, accuracy, coverage, robustness, or another appropriate measure.
- Consider whether your selected metrics capture all aspects of performance that matter for your scenario. What important behaviors might not be fully measured?
- Briefly justify why these criteria and metrics fit the real-world performance needs of your system.

Evaluation criteria

The system uses three sets of criteria to measure the performance of the retrieval layer, the generation layer, and the overall system. At the **retrieval layer** the system should surface the documents which contain the evidence required to answer the query, while minimizing the retrieval of documents which are irrelevant. At the **generation layer** the system should never hallucinate, fabricate an answer, or make unsupported claims, should answer the question as completely and accurately as possible from the retrieved document, should answer the question based only on information in the retrieved documents, should extract citations only from the retrieved documents, should only attribute the citation to the document in which the citation is found, and should only include details in the answer which are relevant to the question. Performance criteria for the **overall system** include an average response latency of no more than 5 seconds per query, an average budget cost of running and judging each query of no more than 10 cents, and appropriately declining to answer the question when the information is not available in the corpus.

Selected metrics

I coded 18 metrics in total: 4 retrieval, 12 generation, and 3 system metrics, as follows. **Retrieval metrics** include recall@k (0 to 1), precision@k (0 to 1), strict recall@k (0 or 1), hit@k (0 or 1). **Generation metrics** include correctness (pass/fail), graded correctness (4 levels 0 to 1), aspect coverage (0 to 1), answer relevance (pass/fail), faithfulness (pass/fail), graded groundedness (0 to 1), citation support rate (0 to 1), chunk attribution (0 to 1), citation precision (0 to 1), unsupported citation rate (0 to 1), quote fidelity rate (0 to 1), and quote attribution accuracy (0 to 1). **Overall system** metrics include latency in seconds, estimated cost in dollars, and refusal outcome (1 of 3 categories). See Capstone\Checkpoint 3.1\rag_evaluation_guide.html and Capstone\Checkpoint 3.1\metric_documentation.md for details.

Potential metric limitations

There are several limitations in what the system currently measures: there are no metrics to evaluate the rank ordering of documents retrieved (are more relevant documents ranked more highly than less relevant documents?); there is also no explicit safeguard metric to measure hallucinations. And there are no measures which account for session context or long running chats which cause degraded responses (at present the system does single question and answer without history, but this should be a future requirement). Also, presently the system calculates an average latency in seconds, but this takes no account of the distribution of response times: what is needed are percentile calculations such as p95 or p99, as users tend to remember the slowest queries the most. Another limitation of the current harness is LLM variability, especially for non-deterministic

metrics calculated by the LLM judge: the test harness should run the test suites n times and examine the spread of non-deterministic metrics to account for variability. Finally, the system does not currently but should also be able to detect anomalous responses which may be the result of prompt injections.

Brief justification for your criteria and metrics

The evaluation system attempts to maintain a balance across retrieval and generation metrics. These layers are interdependent: an answer cannot be generated if the relevant documents are not first retrieved; equally, even when all relevant documents are retrieved, the LLM may not provide an answer to the question the user posed. Given the stochastic and unpredictable nature of Large Language Models, and their proclivity to generate plausible, compelling but fabricated answers, it is vital to build in checks to ensure that all claims are grounded. From the labs we learned that LLMs are prone to backing up their claims with fabricated or misattributed quotes, so the evaluation system includes several metrics to guard against this. The system also seeks a balance between completely automatable, deterministic calculations and LLM-generated metrics that require human review. Finally, the system must exhibit acceptable response times and fall within budget.

Step 3. Testing Approach

Describe the test set and process you used to evaluate the baseline retrieval system.

- Include the number and types of queries or cases used.
- Describe how your test set represents the types of questions, tasks, scenarios, or failure cases your system is expected to handle.
- Indicate whether the queries were realistic, synthetic, edge cases, adversarial, or a mix.
- Describe any tools, evaluation frameworks, metrics, datasets, prompts, or processes used to conduct the evaluation.

Description of your test set

To generate a golden test suite, I used my 15 hand-crafted queries from Checkpoint 2.1 codified as Checkpoint 2.1\golden_suite_2.1.json and added 18 automatically generated queries codified as

`golden_suite.subset.wholecorpus.json`. To emulate the process which Professor Armando has discussed in class, I generated these files as follows.

Firstly, I used my own Anthropic API key and Claude Opus to generate a prompt to instruct an advanced LLM (Fable) to create a golden test suite in JSON format of 100 queries, each with a model response and a list of the most relevant HTML files. The prompt instructed Fable to include questions by information need such as fact lookups, triggers such as assignments, and prior familiarity, according to the Singer/Lemmerich taxonomy. Here is the prompt I used to generate the golden test suite: `golden_suite_generation_prompt.md`.

Secondly, again using my own Anthropic API key, I attempted to feed the complete corpus of 2,419 articles into the prompt, but this would have exceeded the context window, so I ended up extracting a themed subset of 60 articles focused on films and academy awards and provided Fable access to this subset to generate my golden test suite. The Python script to generate the golden test suite is `generate_golden_test_suite.py`. To confirm that every `relevant_files` entry existed on disk and every `supporting_quotes.text` appeared as a substring of the source, I ran a validation script `validate_golden_suite.py`. The resultant suite of 100 generated queries was written to file `golden_suite.json`.

Thirdly, for use in a later checkpoint I also asked Claude Opus to create a supplementary prompt which instructed Fable to create five long-running sequences of questions and answers, again in JSON format, that simulated the typical question and answer sessions for five different profiles of Wikipedia users: assignment researcher, settling an argument, late-night wanderer, trip-planner, media-prompted learner. This suite is designed to check for multi-turn retrieval degradation. The prompt to generate the conversation suite is `conversation_suite_generation_prompt_v2.md`.

Fourthly, so as not to abuse course credits, I used Claude Code to extract 18 representative queries from the 100 in `golden_suite.json`, accounting for each variation of the Singer/Lemmerich taxonomy as well as each type of adversarial query. This produced the file `golden_suite.subset.json`

The fifth step was to use Claude Code to check this subset of queries and related answers and relevant files against the entire Wikipedia corpus of 2,419 files. Only a few tweaks were needed to account for other relevant files and slightly amended answers, resulting in the final golden test file `golden_suite.subset.wholecorpus.json`.

The last step was to ask Claude Code to take my 15 hand-crafted queries from Checkpoint 2.1 and

create a separate golden test suite from these queries, which produced file Checkpoint 2.1\golden_suite_2.1.json.

How the evaluation was conducted

I ran my retrieval script `capstone_checkpoint_3_1_evaluation_solution.py` twice: the first time with `"—input golden_suite.subset.wholecorpus.json"`, the second time with `"—input golden_suite_2.1.json"`. Results were written to the `evaluation_results` folder in json and csv format, including metrics from individual queries and summary metrics. I used Claude Code to report the results of each test and save the report in markdown files `evaluation_results\evaluation_20260916_183901_report.md` and `evaluation_results\evaluation_20260916_190523_report.md`.

As can be seen from the results, the tests covered a broad spectrum of different types of queries according: single-document and multi-document; fact lookups, overview, and in-depth questions; questions simulating users familiar and unfamiliar with the topic; questions simulating different underlying motivations; several adversarial questions, including questions that cannot be answered from the corpus; and the 15 manually created questions from Checkpoint 2.1. The questions were therefore a mix of realistic, synthetic, edge cases, and adversarial queries.

Step 4. Baseline Results

Present the baseline performance results from your evaluation.

- Include scores or findings across your selected metrics. Note any surprising, inconsistent, or difficult-to-interpret results.
- Provide at least two concrete examples of system outputs.
- Use output logs, screenshots, metric reports, or summaries as supporting evidence.

Summary of baseline evaluation results

The automatically generated golden suite of 18 queries had the following key results in the core metrics: 12/18 (66.7%) correct, 15/18 (83.3%) faithful, 18/18 (100%) relevant, 12/18 (66.7%) mean recall, 9.7/18 (53.7%) precision. The handcrafted golden suite of 15 queries had the following key results in the core metrics: 9/15 (60%) correct, 11/15 (73.3%) faithful, 14/15 (93.3%) relevant, 11/15

(73.3%) mean recall, 4.7/15 (31.1%) precision. Average latency was just under 5 seconds, while average cost per query including judging was around 15 cents.

The results illustrate huge room for improvement with the automatic suite failing a third of the tests and the handcrafted failing two fifths. Diving into the reasons, **very poor precision** of retrieved documents stands out: just over one half of top-k results were relevant in the automated suite, and just under one third in the handcrafted suite. In the handcrafted suite 4 of 15 records retrieved none of the required documents (Q003, Q005, Q009, Q014). While the generation side is better, **faithfulness is problematic**: 3 answers out of 18 automated questions made claims that were not supported by the retrieved context, and 4 out of 15 handcrafted; and the **citations** were also problematic. So, both retrieval and generation have issues, which surprised me. After checkpoint 2.1 I was expecting poor retrieval, but I did not expect so many issues on the generation side. **Costs are high** at 5c above budget on average but were anticipated to be high since the retrieval is currently configured to retrieve whole documents and we are currently making five LLM calls per query, including judging.



Example 1: Query, system output, and result

Handcrafted Query Q014: "In the IOC vote that awarded the 1992 Winter Games, which two cities tied and were sent to a run-off, what were their run-off vote totals, and which city was eliminated after the very first round?"

System output: "The provided documents do not contain the answer to this question. They mention the ****1988 Winter Olympics**** host-city vote, not the IOC vote for the ****1992 Winter Games**** [1988_Winter_Olympics.html]."

Result: Fail, recall=0, precision=0

Example 2: Query, system output, and result

Handcrafted Query Q011: "On the day of a 2011 Pacific megathrust quake, the water in several Scandinavian fjords appeared to seethe as if boiling and was caught on film. Which fjord was the most prominent example, and what did scientists conclude after two years of research had caused it?"

System output: "The most prominent fjord was ****Sognefjorden**** [Arctic_Ocean.html]. Scientists concluded that the ****seismic energy of the 2011 Japan earthquake generated seiche waves thousands of kilometers away**** [Arctic_Ocean.html]."

Result: Pass, recall=1, precision=0.33, citation precision=0

Example 3: Query, system output, and result, if applicable

Handcrafted Query Q008: "In the year Ronald Reagan was first elected U.S. president, which player was the first overall pick in the NFL's college draft, and which team chose him?"

System output: "The year Ronald Reagan was first elected U.S. president was 1980, and the first overall pick in the NFL draft that year was Billy Sims, chosen by the Detroit Lions [1980_NFL_draft.html]."

Result: Pass, recall=0.5, precision=0.33, citation support rate=0.67

Example 4: Query, system output, and result, if applicable

Automated Query Q072: "What is the highest-grossing film of all time?"

System output: "According to the documents, ****Avatar**** became the highest-grossing film of all time worldwide [Avatar_(2009_film).html]."

Result: Fail, recall=0, precision=0.33

Example 5: Query, system output, and result, if applicable

Automated Query Q094: "I'm deciding whether to watch a long Western — how does Costner's Horizon project compare in scale and reception to his earlier epic?"

System output: "Costner's ***Horizon: An American Saga*** is described as a Western epic that "will be split into at least four films, each just under three hours in length," with production beginning in August 2022 and the first film released in June 2024 [Kevin_Costner.html]. The project was planned as a series of months-long releases, and the first film's production was expected to last at least 220 days [Kevin_Costner.html].

By comparison, ***Dances With Wolves*** (1990) was a single epic film; it was nominated for 12 Academy Awards and won seven, including Best Picture and Best Director for Costner

[Kevin_Costner.html]. So in scale, *Horizon* is larger as a multi-film project, while *Dances With Wolves* was a standalone epic with major awards success [Kevin_Costner.html].

In reception, *Horizon: An American Saga – Chapter 1* “earned mixed reviews” and “was a box office flop,” and the poor box-office returns were a factor in the indefinite delay of Chapter 2, which also “had received mixed reviews” [Kevin_Costner.html]. *Dances With Wolves*, by contrast, won major Academy Awards and established a much more celebrated reputation [Kevin_Costner.html].”,

Result: Pass, recall=1, precision=0.33, faithfulness=”fail’

Step 5. Performance Analysis

Analyze what the baseline results reveal about your system.

- Identify meaningful strengths and weaknesses.
- Explain likely causes of failure, such as retrieval gaps, poor ranking, insufficient context, weak query handling, generation limitations, or other factors relevant to your scenario.
- Connect your analysis to the evidence presented in Step 4.

Key system strengths

The system has a number of strengths: upward of 60% of answers are correct (66.7% for the automated suite), the answers it produces are largely relevant to the questions being asked (100% for the automated suite, 93.3% for the handcrafted), the vast majority of individual claims are grounded in the retrieved documents (98.4% for the automated suite, 80.7% for the handcrafted), and in most cases the system declines to answer when it is unable rather than fabricating.

Key system weaknesses

However, on multi-document questions in particular, the precision of retrieved documents drops considerably. And on occasion recall fails completely: for several questions, none of the required documents was retrieved. This is the key weakness but not the only one: in some cases, the answers are not entirely faithful to their supporting documents, citations are sometimes misattributed, some answers are incomplete even when the information is available in the retrieved context, some answers make no allowance for the possibility of stale information, and on at least one occasion the

system ignored the instruction to use only retrieved documents and answered from parametric memory.

Likely failure modes and causes

Low precision is caused by a small signal to noise ratio: the retriever is retrieving noisy or irrelevant documents. This may be due to a keyword hit via BM25 somewhere in the document. Currently whole articles are being searched, and many articles are very long. The units of search need to be more granular. Currently TOP_K is set to 3 so if the required documents are crowded out by other thematically adjacent documents that match keywords in the query, then the required information never gets to the answerer LLM. This is the principal failure mode.

For example, Handcrafted Query Q014 failed because it needed 1992_Winter_Olympics.html but instead retrieved 1988_Winter_Olympics.html, 1988_Summer_Olympics.html, and Australia_at_the_Olympics.html. More granular units of retrieval via text splitting and chunking will likely help here, but so too will preserving headings during extraction, or possibly even implementing a special retriever to match against file names, not just content. Splitting multi-topic queries into sub-queries and fusing the responses might also help.

Another example is Automated Query Q072 which failed because it failed to retrieve "Titanic_(1997_film).html", instead retrieving "Harry_Potter_(film_series).html" and "List_of_highest-paid_film_actors.html", in addition to "Avatar_(2009_film).html". As a consequence its answer included no mention of the Titanic and also no caveat about the possibility that its retrieved documents might be out of date. Failure to recognize that documents may be out of date could possibly be rectified by adding instructions to the system prompt.

It is not clear why quotes are being misattributed to the wrong documents. For example, Handcrafted Query Q011 did retrieve the document it needed for its answer, "2011_Tōhoku_earthquake_and_tsunami.html", but it also retrieved "Arctic_Ocean.html" and "Litke_Deep.html" and ended up misattributing "Arctic_Ocean.html" to support its claims. One avenue to explore is adjusting the citation instructions in the system prompt to cite line number or section within the document, not just the document itself.

Ensuring that the LLM does not resort to parametric memory may require experimenting with the instructions in the prompt. For example, in Handcrafted Query Q008, the system needed to retrieve "Ronald_Reagan.html" and "1980_NFL_draft.html" but ended up retrieving "Philadelphia_Eagles.html", "1980_NFL_draft.html", "Washington_Commanders.html". Nowhere in

these documents is it stated that Ronald Reagan was elected president in 1980, so the LLM must have obtained this information from its own parametric memory. Perhaps instructing the LLM to provide its precise source (article, section, line number) for all claims would force it to look only in retrieved documents.

Failures of faithfulness may be down to the temperature setting for the LLM, which is currently set to 0.2. For example, Automated Query Q094 gave a technically correct answer but failed the faithfulness test because one of its assertions, “Dances with Wolves established a much more celebrated reputation”, was not grounded in any of the documents it retrieved ("Kevin_Costner.html", "Clint_Eastwood.html", "Christopher_Nolan.html").

Failure to complete an answer even when all the information has been retrieved occurred with Automated Query Q058. The response stopped after establishing De Niro’s Best Supporting Actor win and the Italian-language detail, never mentioning that Brando won Best Actor for the same character in the first Godfather film. This may be a case of details being “lost-in-the-middle” of long documents so potentially fixable with smaller chunks, and a higher TOP_K and CANDIDATE POOL.

Step 6. Evaluation Framework Validation and Reliability

Demonstrate whether your evaluation framework can reliably detect degraded, manipulated, or otherwise problematic outputs.

- Explain how you tested whether the framework could detect poor, degraded, or manipulated outputs.
- Provide evidence from the validation test.
- Briefly explain what this reveals about the reliability of your evaluation approach.
- Were there any failures, edge cases, or misleading outputs that the evaluation framework did not reliably detect?

How you tested the evaluation framework

The test suites I used involved a mixture of deterministic metrics and LLM Judge metrics. They mixed in queries designed to induce bad behavior, unanswerable from the corpus (Q010, Q012, Q015), temporally stale facts (Q036), ambiguous entities (Q098), and multi-hop/paraphrased questions likely to cause wrong citations or overconfident synthesis. Metrics and judgments were spot checked against the articles themselves for accuracy.

Evidence that the framework detected poor, degraded, or manipulated outputs

On the whole, the evaluation framework worked well e.g.

Q011: correct facts, but cited [Arctic_Ocean.html] instead of the actual source; citation_precision scored it 0.0 and refusal_outcome independently flagged hallucinated, catching a citation attribution bug that faithfulness alone missed.

Q008: Reagan's 1980 election stated correctly but cited to 1980_NFL_draft.html, a document that doesn't support it; citation_support_rate correctly marked that specific claim unsupported while passing the claims the document actually did support.

Q015: a confidently worded but ungrounded inference on an unanswerable question; faithfulness was a fail.

Q072 (full-corpus run): retrieval drifted, and the model asserted "Avatar became the highest-grossing film" without a hedge; this was correctly flagged as a fail.

Q094: an unsupported editorial claim ("X established a more celebrated reputation") was flagged as grounded="false" in both independent reports; the same defect reproduced across two runs, which is strong evidence this is real detection, not noise.

Brief interpretation of the validation results

The framework on the whole caught the failure modes it was built to catch and did so reproducibly; the same defects recurred identically across one run against the limited film and academy awards corpus and two runs against the full corpus. No single metric was trusted alone, the overlap of separate metrics provided credibility.

Potential blind spots in the evaluation framework

However, there were several failures which the evaluation framework failed to catch reliably:

On occasion there were false positives such as Q091 ("tell me about Ben Kingsley's career") which was tagged as "hallucinated" in two separate full corpus runs despite being complete, accurate, and fully cited. Honest refusals which correctly state "the documents don't say X" are currently penalized by the LLM Judge as grounded="false", since no document explicitly asserts an absence; this is a misunderstanding by the LLM which therefore punishes the exact honest behavior the evaluation

framework should reward. The actual answer LLM is also inconsistent, sometimes refusing an unanswerable question, but sometimes guessing (Q015): in this case the judge caught the bad output, but it cannot prevent the underlying system from being inconsistent.

This is the major problem with using LLMs: their stochastic nature means we probably need to run several tests and average the results or take the median result, but even then, there is a certain level of unpredictability which makes complete automation impossible and requires human in the loop to review results for accuracy, which is time-consuming.

Step 7. Reflection and Next Steps

Reflect on the limitations of your current evaluation approach and how you would refine it before using it to guide system optimization.

- Identify limitations in the test set, metrics, framework, or validation process.
- Propose specific improvements you would make next.
- Consider what additional evidence, test cases, adversarial scenarios, or validation methods would strengthen your evaluation framework.

Limitations of your current evaluation approach

I spent a lot of time automating the creation of a golden test suite, so that I could cover the broad spectrum of queries that users are likely to make. But as it turns out the queries that I handcrafted were more effective at pointing out weakness in the retrieval system. And as a developer I am limited in my own imagination about how such a system would be used in practice, so it would be important to build in an audit feature to record queries and analyze and store their retrieval outcomes so that these can be examined back in the lab. Some kind of field testing period would be ideal rather than a public test, where security and privacy concerns would be involved. This is what I have built into a chatbot I built for my work, and I store queries and retrieval paths and results in Azure blob storage for later examination in the lab.

Nevertheless, on the whole, the test set was fairly comprehensive, and it exposed several weaknesses in the retrieval system already. The evaluation framework could be improved by having an LLM paraphrase the queries in the set, which would generate a further set for testing, as we did in the lab 3.1. And multiple test runs would also check for LLM variability, both in retrieval/answering and in judging.

Specific improvements or next steps

Certain metrics need to be further tested and refined, such as refusal outcome and chunk attribution. In addition, I need to create a separate metric for latency of retrieval+answer and with 95% and 99% percentiles, since right now latency includes the LLM judge calls, and the system only calculates the average.

Supporting Materials

Use this section to list or link to any supporting materials included with your submission.

Supporting material	Location, link, or note
Sample test queries	
Output logs, screenshots, or metric reports	
Code, notebook, repository link, or screenshots of setup	

Supporting material	Location, link, or note
Other supporting evidence	
Validation evidence (optional)	

End of Required Capstone Checkpoint 3.1

Return to the assignment page in the learning platform to upload and submit your completed capstone checkpoint and complete any remaining items in this module.

